



BSc (Hons) in **Information Technology**
Specialized in AI

IT3012 : Intelligent Agents

Year 3 · Semester 1 · 2026 Semester 2

Faculty of Computing

Practical 02

Objective & Lecture Mapping: In Lecture 03, we learned that a mathematically perfect "Table-Driven Agent" is impossible to build due to combinatorial explosion. Instead, we must write Agent Programs. Today, you will implement a **Simple Reflex Agent** using Condition-Action rules, observe its fatal flaws in a partially observable environment, and then upgrade it to a **Model-Based Agent** that maintains an internal memory state.

Part 1: Practical Implementation — Coding the Architectures

Step 1.1: Setting the Trap (Partial Observability)

To see why Simple Reflex agents fail, we must handicap your agent's sensors. We are moving from a Fully Observable world to a Partially Observable one.

1. Open your `visual_grid_game.py` from Lab 01.
2. Locate the `get_percept(self)` method.
3. Modify it so it **no longer returns** the exact global coordinates (`agent_pos`). Instead, it should only return local booleans, e.g., `{ 'wall_ahead': True/False, 'food_here': True/False }` based on checking the adjacent cell in its current facing direction.

Step 1.2: The Simple Reflex Agent (Implementation & Failure)

1. Create a new class called `SimpleReflexAgent` .
2. Implement a `sense_and_act(self, percept)` method that uses strictly IF-THEN logic (Condition-Action rules). *Do not use an `__init__` method to store history.*

Example Logic: IF food_here THEN suck; IF wall_ahead THEN turn_left; ELSE move_forward

3. Run the simulation. **Observe:** Watch the agent get trapped in a corner or a U-shaped wall, infinitely repeating a cycle (e.g., turn left, step forward, hit wall, turn left...).

Step 1.3: The Model-Based Agent (Memory & State)

1. Create a new class called `ModelBasedAgent`.
2. Implement an `__init__(self)` method to initialize an internal memory state (e.g., `self.visited_cells = set()` or a relative movement tracker).
3. In `sense_and_act(self, percept)`, first **update the state** (Transition & Sensor Model) by recording the current percept and the last action taken.
4. Update your IF-THEN rules to query the memory.
Example Logic: IF wall_ahead AND left_is_visited THEN turn_right
5. Run the simulation. **Observe:** The agent should now remember where it has been, realize it is in a loop, and choose an alternate path to escape.

Part 2: Theoretical Evaluation & Lecture Mapping

Based on your practical observations above, answer the following questions to map your code directly to the architectural theories covered in Lecture 02.

1. (Remember) According to Lecture 02, why is it impossible to program a mathematically perfect "Table-Driven Agent" for complex environments like Chess? What happens as the agent's lifetime increases?

It is impossible due to combinatorial explosion. For a mathematically perfect Table-Driven Agent, you must pre-compute and store an action for every possible percept sequence. In complex environments like Chess, the number of possible states is astronomically high. As the agent's lifetime T increases, the size of the required table grows exponentially ($|P|^T$, where P is the set of possible percepts). The physical storage required would exceed the number of atoms in the universe, making the table practically impossible to build.

2. (Understand) Look at the code you wrote for your `SimpleReflexAgent` . Identify and explain the specific lines of code that represent the "Condition-Action Rules" discussed in the lecture.

These lines represent strict IF-THEN condition-action logic. The agent directly maps a current, instantaneous sensory percept condition (wall_ahead) to a hardcoded action ('Up' or 'Right'). It does this reflexively without any reference to past history, memory, or the global state of the game.

3. (Analyze) Your `SimpleReflexAgent` likely got stuck in an infinite loop during Step 1.2. Based on the lecture, analyze exactly why this happened. How did the combination of "Partial Observability" and a lack of "Percept History" cause this failure?

The agent gets trapped because it cannot distinguish between different game states that "look" identical to its limited sensors. Due to Partial Observability, the agent only knows if there is a wall directly in front of it; it doesn't know its absolute coordinates or the layout of the maze. Because it lacks Percept History (internal memory), it cannot remember that it was just in this exact spot a moment ago, or that its previous actions didn't help it escape. Consequently, it endlessly triggers the exact same condition-action rule whenever it hits the wall, causing a fatal infinite loop.

4. (Evaluate) In Step 1.3, you added an internal state to your `ModelBasedAgent` . Evaluate how your specific code handles the "Transition Model" (how the world evolves) and the "Sensor Model" (how the agent's actions affect the world).

Transition/Sensor Model: In the code, the `update_state(self, action, percept)` method handles these models. The agent uses its `last_action` to update its internal, estimated position (`self.agent_pos`). By incrementing its internal coordinates based on the direction it chose, it is utilizing a basic Sensor Model (predicting how its own movement actions change its position in the world).

Memory State: It then records these calculated coordinates into a `visited_cells` set.

Evaluation: By tracking this internal state, the agent's IF-THEN rules are upgraded. It can now look at adjacent cells and check `next_pos` not in `self.visited_cells` before moving. This allows the agent to recognize when it is about to enter a loop, intentionally avoid previously visited tiles, and successfully navigate out of U-shaped traps.



Finalize and Lock Lab 02 Documentation



FACULTY OF COMPUTING